on the device. It would decouple the way you write your application either on your device or the cloud.

Messaging bus gives you a reliability guarantee. Most of these approaches are backed up by storage. This allows you to leverage storage locally on the disk, on your gateway, on your device, while your cloud connection is not possible and while you are disconnected from the networks. A messaging based approach also solves the reliability side of the challenges.

Then there are a whole bunch of IoT based platforms. One thing common in all of these platforms is that they all communicate through MQTT and there are many generic faces to their communications. Here you can really bridge it on the IoT platforms on your cloud.

When you have to talk to many of such clouds then, with the help of asynchronous decoupled approach, you can plug in data mappers, which can do the translation from the data format that you want to play around on the device to the format acceptable to a specific cloud. For instance, AWS has its own data model where you don't have to couple your data model on the device with AWS's domain model on the cloud. You can simply decouple it and write translations from your own proprietary formats to the AWS data domain model through a data mapper.

This can translate any of the formats into a cloud IoT specific format. That means that now you have the flexibility to be able to communicate and bring in the services which you want to use from any of the cloud vendors.

There is a whole bunch of infrastructural work that needs to be done for each device. You have to do firmware management locally on the device, you need to do the software management, updates across various different package managers, containers, etc. All of that can be brought to you in a generic way. All the translation and communication are being taken care of by other layers.

You can expose them such that they could be working with any of the IoT vendors. With a small layer of your bridge that can handle a platform-specific operation, you can really make use of a whole bunch of generic features like configuration management, log management, and firmware management that can be done there.

It can all be exposed contextually for a given IoT platform, so that those operations can be accessed remotely. These also solve most of your challenges with respect to deviceOps, because now you can roll out a business logic which you have built continuously.

As a developer you then want to have some tools around it that you can play around with certain logics, components, and modules, locally, and combine them together and take care of them. Your business logic also needs to run there, and because of that you need an open ecosystem that can host your application, so that your application can be bootstrapped and be running. There you can host your CEP algorithms which are working on a time series data.

Now you get an architecture where you have synchronous communication along with a built-in set of functionalities from an infrastructure device management perspective. You have the ability to talk to any cloud, capability to remotely do any operation on the device from any of the cloud vendors, and a way to host any of these applications as you wish. This is what we call thin edge.

Thin edge is an open source project licensed by Apache 2.0, and it can work on almost every hardware. This is our take on resolving these complex challenges.

The advantages of thin edge

Following are some of the advantages of thin edge:

Freedom of choice. You can run thin edge on any language, such as C, C#, or Rust. Because of the asynchronous data mappers approach, the protocol that you use on the device does not need to be what it has to be on the cloud. So, you can get your own custom protocol running right on the device.

Edge runs on any hardware like ARM, docker container modules, etc. A big chunk of your work can be managed on your device from a software management point of view, firmware management point of view, and configuration management point of view.

Robust device management. We already have firmware management with big capabilities like the whole vendor IO approach. There are other generic applications added. We also have software management capabilities. So, if you have your packages written in SNAP, it does not matter. If you want to provide a cryo container, you can do that.

Finally, to highlight some of the important aspects of it, this framework is written in Rust programming language, which is a great programming language because it provides best compile time guarantees and is also great at memory management. Out of the box, Apache 2.0 supports Azure IoT and Amazon Web Services (AWS). It works on almost all the Linux based distros.

Most importantly, Apache 2.0 has a very vibrant community. All of the connectivity software, like protocol drivers and smart PLCs, have already been put in place by the community. EFY

This article is based on a tech talk by Roshan Kumar, Vice President, Software AG, at India Electronics Week 2022. It has been transcribed and curated by Laveesh Kocher, a tech enthusiast at EFY, who has a knack for open source exploration and research